**ACCESO A DATOS - 2.º DAM**

# UP 1 – Ficheros – Serialización de objetos

Guardar y recuperar objetos completos en Java

## 1. Introducción: ¿qué es la serialización?

En la tarea `UP1_TF1` guardamos información de los videojuegos en un fichero de texto (`videojuegos.txt`). Utilizamos `FileWriter`, `FileReader`, `BufferedWriter` y `BufferedReader`, y escribimos el resultado del método `toString()` de cada videojuego. Es decir, guardábamos texto que representaba sus datos, pero no el objeto Videojuego como tal.

Ahora aprenderemos una forma diferente de conservar información: la serialización. Permite transformar un objeto Java completo en bytes y guardarlo en un fichero binario, por ejemplo videojuegos.dat. Después, mediante la deserialización, Java lee esos bytes y reconstruye un nuevo objeto con los datos guardados. También podemos serializar una lista de objetos.

> **IDEA CLAVE**
> Serializar consiste en transformar un objeto en una secuencia de bytes para guardarlo en un fichero. Deserializar es leer esos bytes y reconstruir el objeto.

### 1.1. ¿Qué ocurre exactamente cuándo serializamos un objeto?

Al serializar, Java convierte la información del objeto en una secuencia de bytes y la escribe en un fichero. Al deserializar, lee esos bytes y crea un nuevo objeto en memoria con los datos que habíamos guardado.

Por ejemplo, podemos guardar un videojuego llamado Tetris, cerrar el programa y recuperarlo al día siguiente. No es el mismo objeto en memoria: es un objeto nuevo que conserva los datos guardados.




El fichero queda guardado en el disco, aunque se cierre el programa. Al leerlo, Java crea otro objeto con los datos almacenados

> **IDEA CLAVE**
> El fichero `.dat` contiene datos binarios: no está pensado para leerse con un editor de texto. Utilizaremos `ObjectOutputStream` para guardar objetos y `ObjectInputStream` para recuperarlos.

## 2. La interfaz `Serializable`

Para permitir que Java guarde un objeto de nuestra clase, debemos indicar que esa clase se puede serializar.

Se hace añadiendo `implements Serializable`. `Serializable` es una interfaz especial de Java que no contiene métodos que tengamos que implementar.

Su función es indicar a Java que los objetos de esa clase pueden serializarse, es decir, convertirse en bytes para guardarlos o transmitirlos.

> **IDEA CLAVE**
> Serializable es un caso especial: no exige implementar ningún método, simplemente marca la clase como serializable.

```
import java.io.Serializable;

public class Videojuego implements Serializable {
    private static final long serialVersionUID = 1L;

    private int id;
    private String titulo;
    private double precio;

    public Videojuego(int id, String titulo, double precio) {
        this.id = id;
        this.titulo = titulo;
        this.precio = precio;
    }

    @Override
    public String toString() {
        return id + " - " + titulo + " - " + precio + " €";
    }
}
```

Nos fijamos en dos cosas:

- **implements Serializable**: indica que los objetos de esta clase se pueden guardar en un fichero mediante serialización.
- **serialVersionUID = 1L**: identifica la versión de serialización de la clase. Cuando guardamos un objeto, Java almacena también este identificador y, al recuperarlo, comprueba que coincide con el de la clase actual.

El valor puede ser 1L, 2L, 3L u otro número de tipo long. La letra L indica precisamente ese tipo de dato. No es necesario empezar por 1 ni aumentar el número cada vez que modificamos la clase.

De momento utilizaremos 1L y no profundizaremos en los cambios de versión.

## 3. Escribir un objeto en un fichero

Utilizamos `ObjectOutputStream` y el método `writeObject()`. Para conectar el flujo al fichero, empleamos `FileOutputStream`. `try-with-resources` cerrará el flujo automáticamente.

```
Videojuego juego = new Videojuego(1, "Tetris", 19.99);

try (ObjectOutputStream salida = new ObjectOutputStream(
        new FileOutputStream("videojuego.dat"))) {
    salida.writeObject(juego);
    System.out.println("Videojuego guardado");
} catch (IOException e) {
    System.out.println("Error al guardar: " + e.getMessage());
}
```

¿Qué hace cada instrucción?

- `FileOutputStream` abre el fichero;
- `ObjectOutputStream` prepara la escritura de objetos;
- `writeObject(juego)` guarda los datos del objeto. Si el fichero ya existía, este ejemplo lo sobrescribe.

## 4. Leer un objeto del fichero

Para recuperar el objeto utilizamos `ObjectInputStream` y `readObject()`. Como `readObject()` devuelve un `Object`, debemos convertirlo a `Videojuego`.

```
try (ObjectInputStream entrada = new ObjectInputStream(
        new FileInputStream("videojuego.dat"))) {

// Es necesario hacer el casting → (Videojuego), porque el método readObject()

// devuelve un objeto de tipo Object, no de tipo Videojuego.

    Videojuego recuperado = (Videojuego)entrada.readObject();
    System.out.println(recuperado);
} catch (IOException | ClassNotFoundException e) {
    System.out.println("Error al leer: " + e.getMessage());
}
```

## 5. Guardar y recuperar una lista de objetos

Hasta ahora hemos aprendido a guardar y recuperar un único objeto Videojuego. Pero ¿qué ocurre si tenemos varios videojuegos?

En el proyecto LudotecaIES trabajastéis con un catálogo que contiene varios objetos Videojuego. En lugar de guardar cada objeto por separado, podemos almacenar una lista completa utilizando ArrayList.

La ventaja es que podemos guardar todos los videojuegos con una sola llamada a **writeObject()**.

### 5.1. Guardar una lista de videojuegos

Primero creamos una lista y añadimos dos videojuegos:

```
ArrayList<Videojuego> juegos = new ArrayList<>();
juegos.add(new Videojuego(1, "Tetris", 19.99));
juegos.add(new Videojuego(2, "Pac-Man", 14.99));
```

En este momento, los dos videojuegos se encuentran en la memoria del programa, dentro del ArrayList.

Ahora guardamos la lista completa en el fichero videojuegos.dat:

```
try (ObjectOutputStream salida = new ObjectOutputStream(
        new FileOutputStream("videojuegos.dat"))) {
    salida.writeObject(juegos);
    System.out.println("Lista de videojuegos guardada");
} catch (IOException e) {
    System.out.println("Error al guardar: " + e.getMessage());
}
```

### ¿Qué ocurre aquí?

- `FileOutputStream` abre el fichero para escribir.
- `ObjectOutputStream` permite serializar los objetos.
- `salida` es la variable que representa el flujo de escritura.
- `salida.writeObject(juegos)` guarda la lista completa, incluidos los videojuegos que contiene.

No necesitamos recorrer la lista con un bucle ni escribir cada videojuego individualmente.

> **IMPORTANTE**
> ArrayList ya implementa Serializable, pero los objetos que contiene también deben poder serializarse. Por eso nuestra clase Videojuego debe implementar esta interfaz.

### 5.2. Recuperar la lista de videojuegos

Una vez guardada la lista, podemos leer el fichero y reconstruirla en memoria.

```
try (ObjectInputStream entrada = new ObjectInputStream(
        new FileInputStream("videojuegos.dat"))) {
    ArrayList<Videojuego> recuperados =
            (ArrayList<Videojuego>) entrada.readObject();
    for (int i = 0; i < recuperados.size(); i++) {
      System.out.println(recuperados.get(i));
}
} catch (IOException | ClassNotFoundException e) {
    System.out.println("Error al recuperar: " + e.getMessage());
}
```

### 5.3. ¿Por qué aparece un aviso en el casting?

Es posible que IntelliJ muestre una advertencia en esta línea:

```
(ArrayList<Videojuego>) entrada.readObject();
```

El aviso se denomina *unchecked cast (conversión no comprobada).*

Esto ocurre porque Java puede comprobar que el objeto recuperado es un ArrayList, pero no puede verificar completamente que todos sus elementos sean de tipo Videojuego durante esa conversión.

> **IDEA CLAVE**
>
> Con writeObject() podemos guardar una lista completa de objetos en un fichero binario.
>
> Posteriormente, readObject() permite recuperar esa lista con todos sus elementos, sin tener que reconstruir cada videojuego manualmente

## 6. ¿Qué ocurre si modificamos la clase después?

Imagina que guardamos varios videojuegos y, posteriormente, cambiamos el tipo de precio de double a int. Puede que los datos antiguos ya no sean compatibles con la nueva clase. En esos casos, la deserialización puede fallar.

El campo serialVersionUID ayuda a comprobar la compatibilidad entre versiones, pero no convierte automáticamente los datos ni soluciona cualquier cambio. Por ejemplo, cambiar el tipo de un atributo puede seguir causando errores aunque mantengamos el mismo identificador.

> **PARA RECORDAR**
>
> Si modificamos la estructura de la clase, debemos comprobar que los ficheros antiguos todavía pueden leerse. No cambiaremos sus tipos durante la práctica.

## 7. ¿En qué se diferencia de RandomAccessFile?

| RandomAccessFile | Serialización de objetos |
|---|---|
| Permite desplazarnos a posiciones concretas en bytes. | Guarda o recupera objetos completos. |
| Podemos modificar un campo en una posición conocida. | Para actualizar el catálogo, lo habitual es cargarlo y volver a guardar la lista. |
| Tenemos que definir el orden y tamaño de los datos. | Java gestiona automáticamente la representación de los objetos serializados. |

## 8. Resumen

| Concepto | Explicación |
|---|---|
| **Serialización** | Convierte un objeto en bytes para almacenarlo. |
| **Deserialización** | Reconstruye un objeto a partir de los datos del fichero. |
| **Serializable** | Interfaz que debe implementar la clase de los objetos que queremos guardar. |
| **ObjectOutputStream** | writeObject() permite guardar objetos. |
| **ObjectInputStream** | readObject() permite recuperarlos. |
| **serialVersionUID** | Identificador de versión que ayuda a comprobar la compatibilidad. |